

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Image Registry & Distribution Data _____
Class 24 Applied Computing Name Zhou Jiaqi ID 202483910036

1. Experimental Purpose

- a) **Practice Version Control:** I learned to use Docker Tags to manage different iterations of a cloud application.
- b) **Validate Decoupled Deployment:** I simulated a disaster recovery scenario to verify the ability to deploy services in a remote environment without source code.
- c) **Security Best Practices:** I implemented Secret Management using Personal Access Tokens (PAT) instead of plaintext passwords.

2. Experiment Contents

- i. In this lab, I transitioned from a "Standalone Developer" to a "Cloud Engineer."
I:
- ii. Configured secure authentication with GitHub Container Registry (GHCR)
- iii. Tagged and pushed web application image to the cloud
- iv. Performed a version upgrade (v1.0 to v2.0) and observed the Layered Storage mechanism
- v. Performed a "Clean-room Deployment" by pulling image onto a fresh environment after wiping local data

3. Detailed Steps of the Experiment

Step 1: Secure Authentication (PAT Generation)

Cloud platforms use Tokens for automation and security.

1.1 Generate Token

I navigated to GitHub Settings -> Developer settings -> Personal access tokens -> Tokens (classic).

1.2 Permissions

I clicked "Generate new token (classic)" and selected the scope write:packages (this

automatically includes repo and read:packages).

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☒ write:packages	Upload packages to GitHub Package Registry
☒ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ write:org	Read and write org and team membership, read and write org projects
☐ read:org	Read org and team membership, read org projects
☐ manage_runners:org	Manage org runners and runner groups

Figure 1: PAT Permissions Configuration

1.3 Save Token

I copied the generated token immediately. GitHub will not display it again after this session.

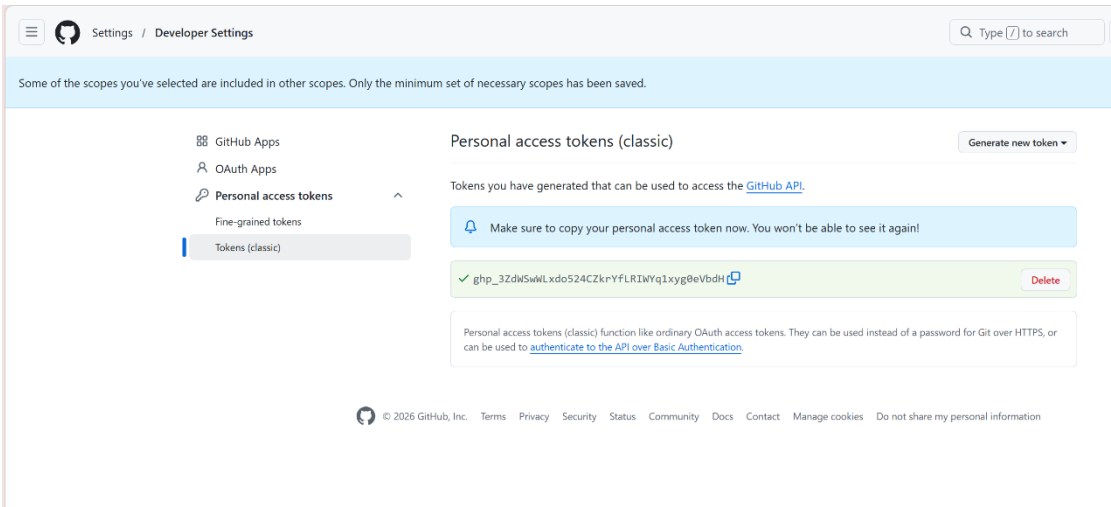


Figure 2: Generated Personal Access Token

1.4 Login via Terminal

In the Codespaces terminal, I ran the following code:

```
问题 输出 调试控制台 终端 端口
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ export CR_PAT=ghp_3ZdwSwWLxdo524CZkrYfLRIWYq1xyg0eVbdH
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ echo $CR_PAT | docker login ghcr.io -u Puffkten-zjq --password-stdin

WARNING! Your credentials are stored unencrypted in '/home/codespace/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

Figure 3: Docker Login with PAT

Observation: I saw "Login Succeeded" upon successful authentication.

Note: A warning about "credentials stored unencrypted" appeared, which is normal in a Cloud IDE environment like Codespaces.

Step 2: Tagging for the Cloud

Docker images require a specific naming convention to be recognized by cloud registries.

2.1 Build the Docker Image cloud-lab-image:v1 based on lab 2.

```
问题 输出 调试控制台 终端 端口
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v1 .
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 0.6s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.2s
=> => extracting sha256:abaae85d1626e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2 0.2s
=> => extracting sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 0.0s
=> => extracting sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 0.0s
=> => extracting sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 0.0s
=> => extracting sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697 0.0s
=> => extracting sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd 0.0s
=> => extracting sha256:fbaed3f7fcb6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18 0.5s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html 1.1s
=> exporting to image 0.2s
=> => exporting layers 0.1s
=> => exporting manifest sha256:a3b4bb7680b3647ec687e90a356e1f884c818ba4c1399b96bd0e0eec68a57c75 0.0s
=> => exporting config sha256:0a0c99b1f803ce36bc53da6d4421812c5cf15aae7cefcf3ee674c1c67ea018da 0.0s
=> => exporting attestation manifest sha256:6eb26b49ef784e0bf983402b283f18040ee672171de9324640b052e8ba95 0.0s
=> => exporting manifest list sha256:edb23c48509f8238e474b20131cdf39542d35a72170beecc28a17933fbd0422 0.0s
=> => naming to docker.io/library/cloud-lab-image:v1 0.0s
=> => unpacking to docker.io/library/cloud-lab-image:v1 0.0s
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cloud-lab-image:v1	edb23c48509f	92.7MB	26MB	

Figure 4: Building v1.0 Docker Image

2.2 Apply Tag: Convert local image into a cloud-ready format.

Note: GitHub username must be in LOWERCASE

`docker tag cloud-lab-image:v1 ghcr.io/YOUR_USERNAME/cloud-app:v1.0`

Step 3: Pushing the Image (The "Ship" Phase)

I uploaded the packaged application to GitHub's global infrastructure.

3.1 Push

`docker push ghcr.io/YOUR_USERNAME/cloud-app:v1.0`

```
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/puffkten-zjq/cloud-app:v1.0
The push refers to repository [ghcr.io/puffkten-zjq/cloud-app]
10177b5f919a: Pushed
43f834d60d8a: Pushed
de1b677d8c00: Pushed
a5008f4a4b25: Pushed
2ff453bc7a8d: Pushed
6a0ac1617861: Pushed
abaae85d1626: Pushed
94d083cf706a: Pushed
e654dbbbb9e1: Pushed
fbaed3f7fcb6: Pushed
v1.0: digest: sha256:edb23c48509f8238e474b20131cdf39542d35a72170beecc28a17933fbd0422 size: 856
```

Figure 5: Pushing Image to GHCR

3.2 Verify

I went to my GitHub Profile -> Packages and confirmed that cloud-app was listed.

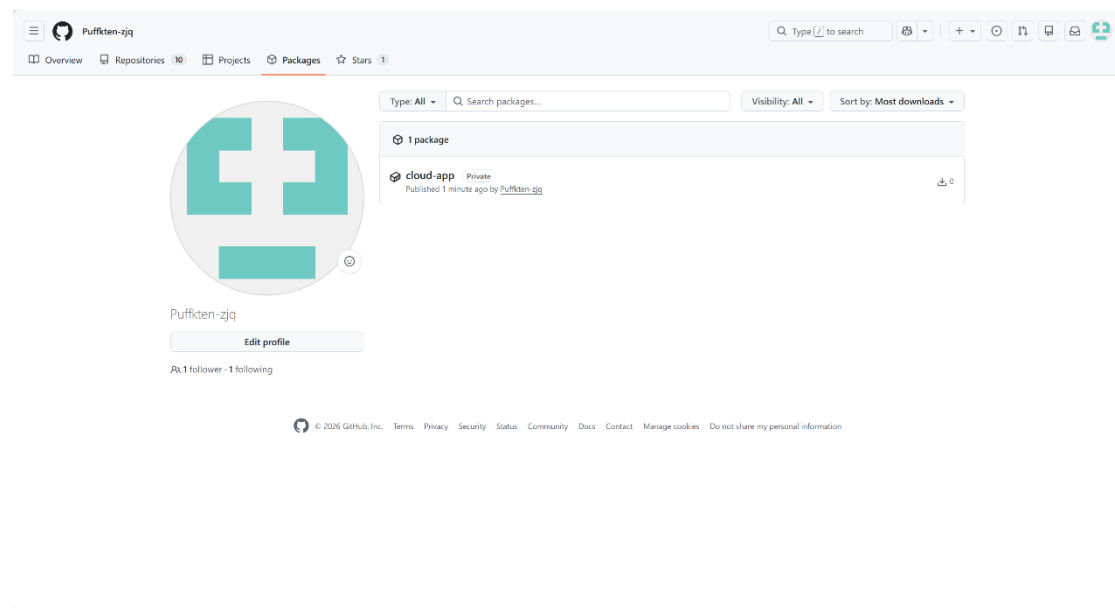


Figure 6: Package Verification on GitHub

Step 4: Iteration & Layer Analysis (v2.0)

Cloud computing relies on incremental updates.

4.1 Modify Code

I opened index.html and changed a visible element, such as updating text to "Welcome to Version 2.0".

```
问题 输出 调试控制台 终端 端口
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v2 .
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 903B
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:62abf049241b67327533cd3e4b558f152ab0187c6ee691226da00e710e65c0a1
=> => exporting config sha256:32c599278e47fb9d8be84390a71931f4c6ce04440f18097516daa5880ddcd391
=> => exporting attestation manifest sha256:6cb11b2b75ce0de225f162f8a1b17e456dd9326510ba9a42f89f8ff867b17b30
=> => exporting manifest list sha256:82ef7404b339b81a0cd56ab17114480fe3f83425950b90d43eac137a01480e4c
=> => naming to docker.io/library/cloud-lab-image:v2
=> => unpacking to docker.io/library/cloud-lab-image:v2
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v2
docker: 'docker tag' requires 2 arguments

Usage:  docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]

Run 'docker tag --help' for more information
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v2 ghcr.io/puffkten-zjq/cloud-app:v2.0
```

Figure 7: Code Modification for v2.0

4.2 Rebuild & Retag

docker build -t cloud-lab-image:v2 .

docker tag cloud-lab-image:v2 ghcr.io/YOUR_USERNAME/cloud-app:v2.0

4.3 Push v2.0

`docker push ghcr.io/YOUR_USERNAME/cloud-app:v2.0`

Observation: I noticed that some lines indicated "Layer already exists." This demonstrates Docker's efficiency—only the changed layer (the HTML) was uploaded, saving time and bandwidth.

```
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/puffkten-zjq/cloud-app:v2.0
The push refers to repository [ghcr.io/puffkten-zjq/cloud-app]
5ad930c057ec: Pushed
16a1c0af70ba: Pushed
43f834d60d8a: Layer already exists
de1b677d8c00: Layer already exists
94d083cf706a: Layer already exists
e654dbbbb9e1: Layer already exists
a5008f4a4b25: Layer already exists
6a0ac1617861: Layer already exists
abaae85d1626: Layer already exists
fbaed3f7fcbe: Layer already exists
v2.0: digest: sha256:82ef7404b339b81a0cd56ab17114480fe3f83425950b90d43eac137a01480e4c size: 856
```

Figure 8: Pushing v2.0 to GHCR

Step 5: Disaster Recovery (The "Pull" Test)

This test proves that the application is independent of the local machine.

5.1 Wipe Local Data:

I deleted all local images and containers to simulate a new server.

`docker system prune -a`

```
问题 输出 调试控制台 终端 端口 ⓘ
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker system prune -a
untagged: ghcr.io/puffkten-zjq/cloud-app:v2.0
deleted: sha256:82ef7404b339b81a0cd56ab17114480fe3f83425950b90d43eac137a01480e4c
deleted: sha256:62abf049241b67327533cd3e4b558f152ab0187c6ee691226da00e710e65c0a1
deleted: sha256:6cb11b2b75ce0de225f162f8a1b17e456dd9326510ba9a42f89f8ff867b17b30

Deleted build cache objects:
w2pt1vbgc3qrtxpjj8vlsnv13
ee6o0zm5tv00y4jytj97ggk2n
8bwsjxvv05fal939n8wxnbyx7
hgw0wvcf1gmgoaif73kfs1pmb
lcyj6wcg96vxjgug4kz8kf0f9
pahao2fhptzmamdc1xmc01hs3
6jiex2d2s8n9sx1hmzimpyuew
ucsh6iu92i2g679rp9xas1vaa
8jppj7bp51smhv7se89d3r2yqd
on1w3r6h9xaqftf83qbtbhto
qg86ztsk91cazyg6atq79luwm
kozvkj7tmo082p5gy6bca319i
wy8ule1su23hqr9s9zisfhaos

Total reclaimed space: 92.83MB
```

Figure 9: Docker System Prune

5.2 Remote Deployment

I deployed directly from the cloud without any local source code files.

`docker run -d -p 8888:80 ghcr.io/YOUR_USERNAME/cloud-app:v1.0`

```

@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/puffkten-zjq/cloud-app:v1.0
Unable to find image 'ghcr.io/puffkten-zjq/cloud-app:v1.0' locally
v1.0: Pulling from puffkten-zjq/cloud-app
10177b5f919a: Pull complete
6a0ac1617861: Pull complete
abaae85d1626: Pull complete
43f834d60d8a: Pull complete
de1b677d8c00: Pull complete
94d083cf706a: Pull complete
e654dbbbb9e1: Pull complete
a5008f4a4b25: Pull complete
fbaed3f7fcbe: Pull complete
2ff453bc7a8d: Download complete
Digest: sha256:edb23c48509f8238e4e74b20131cdf39542d35a72170beecc28a17933fbd0422
Status: Downloaded newer image for ghcr.io/puffkten-zjq/cloud-app:v1.0
8ff5daadb129683787baa35ee8ef382b070f66979ffa69c0d863b3b9904e76a0

```

Figure 10: Remote Deployment with v1.0

5.3 Verify

I accessed the service via the "Ports" tab on port 8888.

5.4 Run v2.0

`docker run -d -p 8888:80 ghcr.io/honglinyuan/cloud-app:v2.0`

I encountered "Bind for 0.0.0.0:8888 failed: port is already allocated."

Step 6: Image Optimization - "Standard vs. Alpine"

In cloud computing, image size directly impacts storage costs and deployment speed.

6.1 Compare Base Images

I created two different Dockerfiles to see the impact of OS selection.

```

@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ cat > Dockerfile.alpine <<'EOF'
FROM nginx:alpine
COPY . /usr/share/nginx/html
EOF
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ cat Dockerfile.alpine
FROM nginx:alpine
COPY . /usr/share/nginx/html

```

Figure 11: Creating Two Different Dockerfiles

6.2 Create a file named Dockerfile.alpine with the following content:

```

@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-app:standard -f Dockerfile .
=> naming to docker.io/library/cloud-app:standard 0.0s
=> unpacking to docker.io/library/cloud-app:standard 0.0s
@Puffkten-zjq → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-app:alpine -f Dockerfile.alpine .
[+] Building 1.0s (7/7) FINISHED
=> [internal] load build definition from Dockerfile.alpine 0.0s
=> transferring dockerfile: 91B 0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 0.3s
=> [internal] load .dockerignore 0.0s
=> transferring context: 2B 0.0s
=> [internal] load build context 0.0s
=> transferring context: 50.23kB 0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a 0.0s
=> resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a 0.0s
=> [2/2] COPY . /usr/share/nginx/html 0.1s
=> exporting to image 0.3s
=> exporting layers 0.1s
=> exporting manifest sha256:11e47d63f9dd8e8402500753ff7a545a00430b3b448b03b663eee0d214f226f7 0.0s
=> exporting config sha256:9b4bb97db9e90e4b10df4ff88e7cfc840eb29fc4185c9f64fcae33138cf29774 0.0s
=> exporting attestation manifest sha256:0cc07038588cf92b0ca36a2aff1f1c47b1fc4e0df90779a328ab6e641659c80a 0.0s
=> exporting manifest list sha256:3ba402bd8c8303ce8903a6f72e646e14dd3f3ce4687f1431e6750a07326f04bd8 0.0s
=> naming to docker.io/library/cloud-app:alpine 0.0s
=> unpacking to docker.io/library/cloud-app:alpine 0.1s

```

Figure 12: Dockerfile.alpine Content

6.3 Build both versions

`docker build -t cloud-app:standard -f Dockerfile .`

docker build -t cloud-app:alpine -f Dockerfile.alpine .

6.4 Size Comparison

I compared the size of cloud-app:standard vs. cloud-app:alpine.

```
问题 输出 调试控制台 终端 端口 1
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ docker pull nginx:latest
b4a248c845e5: Pull complete
7f8b1a2b17d8: Pull complete
45381ecb0e2f: Pull complete
cc5f57206478: Download complete
5e6b66b5e5f1: Download complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ docker pull nginx:alpine
alpine: Pulling from library/nginx
Digest: sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
Status: Downloaded newer image for nginx:alpine
docker.io/library/nginx:alpine
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ docker images | grep nginx
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx:alpine          8b1e78743a03          93.6MB          27MB
nginx:latest         5aca99593157          241MB          66MB
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ docker images | grep cloud-app
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-app:alpine     3ba402bd8c83          93.2MB          26.1MB
cloud-app:standard   405df3385dc7          92.7MB          26MB
ghcr.io/puffkten-zjq/cloud-app:v1.0   edb23c48509f          92.7MB          26MB  U
```

Figure 13: Image Size Comparison Results

Report Task: Record the exact sizes and explain why "Alpine" is the industry standard for production microservices.

Step 7: Use Git to Save Your Changes

git add .

(Gather all lab changes)

git commit -m "Finished Lab 03"

(Seal the package)

git push

(Ship it to GitHub for permanent storage)

```
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ git add .
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ git commit -m "Finished Lab 03"
[main 78da248] Finished Lab 03
 2 files changed, 3 insertions(+), 1 deletion(-)
 create mode 100644 Dockerfile.alpine
@Puffkten-zjq →/workspaces/cloud-computing-lab-github (main) $ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 457 bytes | 457.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Puffkten-zjq/cloud-computing-lab-github
 deba107..78da248  main -> main
```

Figure 14: Git Operations

4. Image Size Analysis Results

Analysis: Alpine-based images are significantly smaller than standard images

because Alpine Linux is designed specifically for container environments. Alpine uses musl libc instead of glibc, and BusyBox instead of traditional GNU coreutils, resulting in a minimal but functional Linux distribution. This smaller size leads to faster cold start times, reduced storage costs, and decreased network transfer times during deployments.

For production microservices, Alpine has become the industry standard because these size savings compound exponentially when deploying applications hundreds or thousands of times across container orchestration systems.

5. Reflection

Question: Why did Step 4.3 (Pushing v2.0) upload much faster than Step 3.1? Relate this to the concept of Layer Caching.

Answer: The significant speed difference stems from Docker's layered architecture and intelligent caching mechanism. When pushing v1.0, all layers must be uploaded to the registry for the first time. However, when pushing v2.0, Docker analyzes each layer and compares it against layers already present in the registry. Since only the HTML file was modified, all other layers already existed in the registry and displayed "Layer already exists" messages. Only the single modified layer containing the updated HTML was transferred, dramatically reducing upload time and bandwidth.

Question: Why is using a Personal Access Token (PAT) more secure than using your actual GitHub password in a terminal?

Answer: Personal Access Tokens offer several security advantages. First, PATs can be granted with specific, limited scopes—just the permissions necessary for a particular task—whereas a password provides full account access. Second, tokens can be easily revoked without affecting the primary account credentials. If a token is compromised, it can be deleted immediately in GitHub settings. Third, tokens leave no trace in shell history when using `--password-stdin`, whereas passwords typed directly into terminals may be captured in command history or log files.

6. Conclusion

This laboratory exercise successfully demonstrated the end-to-end workflow of cloud-based container image management. Through practical implementation, I gained hands-on experience with GitHub Container Registry, Docker image tagging

and pushing, version iteration with layered storage, disaster recovery scenarios, and image optimization.

The experiment proved that containerized applications achieve true portability through cloud registries. The disaster recovery test validated this portability—after completely wiping local images, I successfully pulled and ran the application directly from the cloud.

The Layer Caching mechanism proved to be a powerful optimization. Watching the v2.0 push skip unchanged layers demonstrated how Docker intelligently manages storage and transfer, which is fundamental to efficient DevOps practices.

The image optimization comparison highlighted the importance of base image selection. Alpine-based images offer compelling advantages for production microservices in terms of size, speed, and cost.